Hacettepe University Computer Engineering Department

Experiment 4 - Combinational Circuits in Verilog

Due date: 02/12/2025, 23:59:59 **Via: https://submit.cs.hacettepe.edu.tr/**

BBM233 Digital Design Lab - 2025 Fall

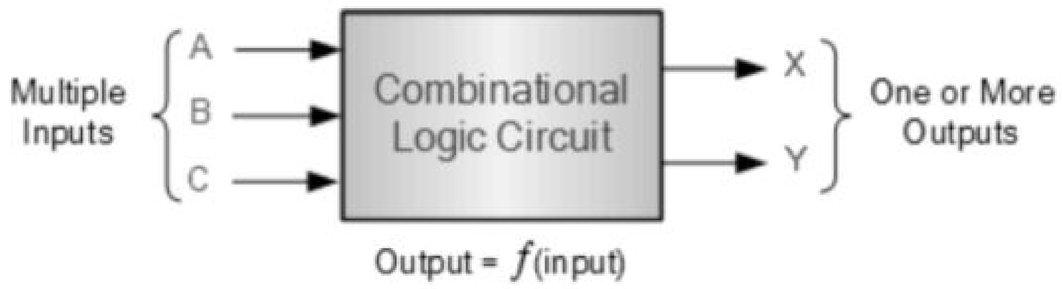
AIM

Designing and simulating combinational circuits in Verilog HDL.

BACKGROUND

Combinational Circuits

Combinational circuits are circuits whose outputs, at any instant of time, depend only on the present inputs (the combinational circuits do not use any memory elements). That is, the previous inputs or state of the circuit do not have any effect on its present state.



A combinational circuit performs a specific information-processing operation fully specified logically by a set of Boolean functions. The 'n' input variables come from an external source whereas the 'm' output variables go to an external destination. In many applications, the source or destination are storage registers.

Combinational Circuit Design Steps

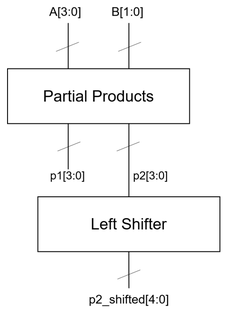
To design a combinational circuit, start with the problem definition and use the following steps:

1. Identify the number of inputs and outputs of the circuit from the given specifications and assign them letter symbols.
2. Derive the truth table for each of the outputs based on their relationships to the inputs.
3. Simplify the Boolean function for each output (e.g. using Karnaugh Maps or Boolean algebra).
4. Construct the logic circuit diagram using Boolean functions obtained from the Step-3.

PLAGIARISM CONTROL NOTICE

Students must implement their solutions individually. All submissions will be submitted to a plagiarism check. Any submissions that show a high level of similarity will be graded with -100 pts and reported as plagiarism attempts to the ethics committee.

Part 0: Multiplier Operations (15%)



Binary multiplication can be performed using the **partial products method**, similar to decimal multiplication. When multiplying two binary numbers, we generate partial products by multiplying (using AND operation) one number with each bit of the other. These partial products are then shifted and added together to produce the final result. Left shifting is a simple and powerful operation in multiplication - shifting left by one position is equivalent to multiplying by 2. Just as we sum individual digits in decimal multiplication, we add the shifted partial products to obtain the final product in binary multiplication.

Circuit Specification

Inputs: One 4-bit and one 2-bit number: $A = A_3A_2A_1A_0$ and $B = B_1B_0$

Outputs: Two partial products:

$$p_1 = A \text{ AND } B_0 \text{ (4-bit)}$$
$$\text{shifted_}p_2 = (A \text{ AND } B_1) << 1 \text{ (5-bit)}$$

Partial Products:

- $p_1 = A$ AND B_0 generates the first partial product
- $p_2 = A$ AND B_1 generates the second partial product
- Each bit of B is replicated 4 times and AND-ed with A
- If $B_0 = 1$, then $p_1 = A$; otherwise $p_1 = 0000$
- If $B_1 = 1$, then $p_2 = A$; otherwise $p_2 = 0000$

Left Shift Operation:

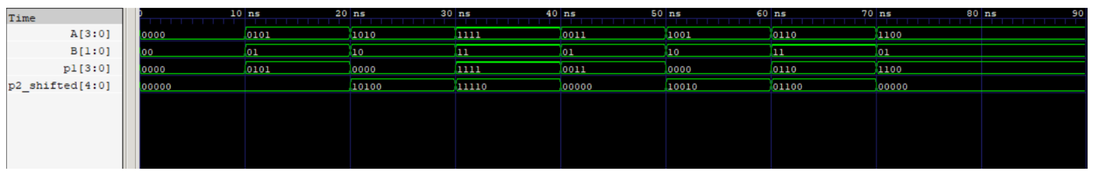
The second partial product p_2 must be shifted left by 1 position (equivalent to multiplying by 2). This produces a 5-bit result where the LSB becomes 0.

Experiment Steps

In this part, you will design a **4-bit left shifter** and **partial product generator** and implement them in Verilog HDL.

- Implement the **left shifter** using **dataflow design approach**. The shifter takes a 4-bit input and produces a 5-bit output by shifting left by 1 position. Use **assign** statements. Save your file as **multiplier.v**.
- Write a testbench for the left shifter and test for various input combinations. Save your file as **multiplier_tb.v**
- **You MUST download and use these starter code files before you start working! Do NOT change the I/O port names, order, or width!**

Make sure to obtain a similar waveform that shows the correctness of your design.



Experiment 4 - Combinational Circuits in Verilog

Due date: 02/12/2025, 23:59:59 **Via: https://submit.cs.hacettepe.edu.tr/**

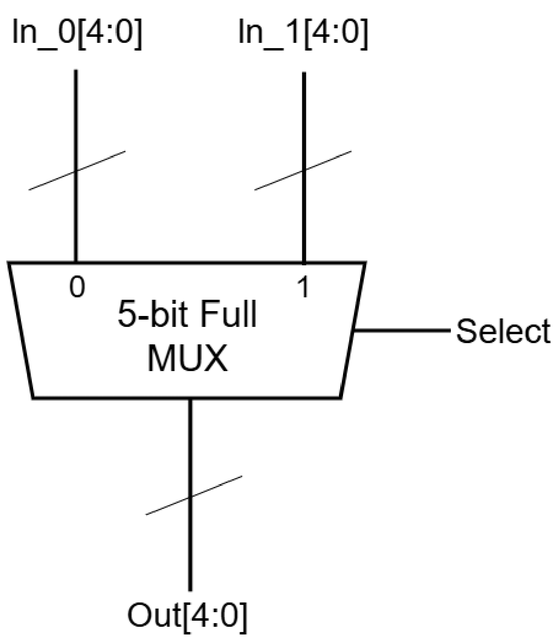
BBM233 Digital Design Lab - 2025 Fall

Part 1 - Multiplexer (20%)

A multiplexer (MUX) is a combinational logic circuit designed to switch one of several input lines through to a single common output line by the application of a control signal. A MUX has a maximum of 2^n data inputs. One of the inputs is connected to the output based on the value of the selection line(s). There will be 2^n possible combinations of 1s and 0s since there are 'n' selection lines.

5-bit 2-to-1 MUX

In a 5-bit 2-to-1 MUX, only one out of two 5-bit wide inputs is selected as the output based on a 1-bit select signal.



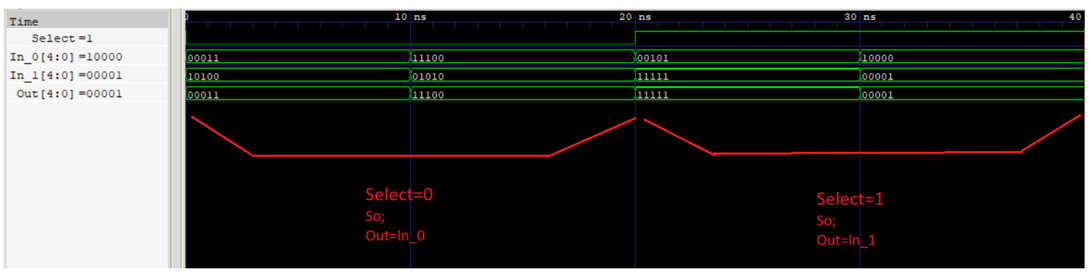
Select	Out
0	In_0
1	In_1

Experiment Steps

In this second part, you will design a **5-bit 2-to-1 MUX** and implement it in Verilog HDL.

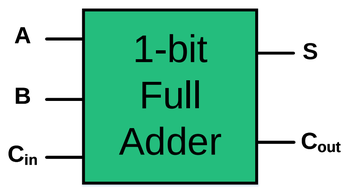
- Using the design specifications given in the truth table, implement a 5-bit 2-to-1 MUX using **data-flow** design approach.. Save your file as **five_bit_2x1_mux.v**
- **You MUST download and use these starter code files before you start working! Do NOT change the I/O port names, order, or width!**

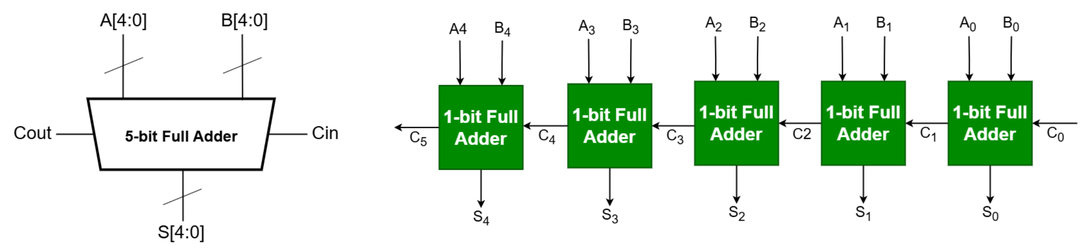
Make sure to obtain a similar waveform which shows the correctness of your design. You should prove that the correct signal is chosen for all possible select signal inputs.



Part 2 - 5-bit Full Adder (25%)

A full adder is a combinational logic circuit that forms the arithmetic sum of three binary numbers. A full adder consists of three inputs and two outputs. Two of the input variables denoted by A and B represent the two numbers to be added. The remaining input variable C_{in} represents the carry from the previous summation. The two outputs of the logic circuit consist of the summation result and output carry C_{out}.





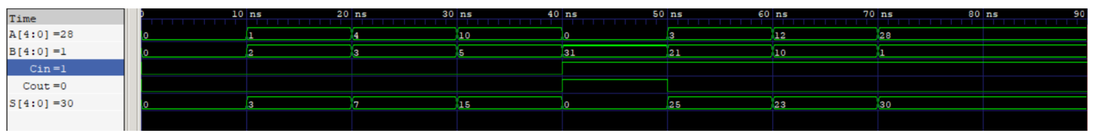
A 5-bit Ripple Carry Adder can be implemented by instantiating four 1-Bit Full Adder modules as shown in the figure above.

Experiment Steps

In this third part, you will design a **5-bit full RCA adder** by using 5 single-bit full adders and implement it in Verilog HDL (check the practice example from the last week - you are allowed to use the solution from the slides for this part):

1. Implement a single-bit full adder using **data-flow** design approach. Save your module as **full_adder.v**. Don't forget to write a testbench and test your module before moving on to the next step.
2. Using 5 single-bit full adders, implement a 5-bit full RCA adder using **structural design approach** with **explicit association**. Save your module as **five_bit_rca.v**.
3. Test your modules by writing an appropriate testbenches **five_bit_rca_tb.v** and **full_adder_tb.v** for all possible input cases.
4. **You MUST download and use these starter code files before you start working! Do NOT change the I/O port names!**

Make sure to obtain a similar waveform that shows the correctness of your design.



Experiment 4 - Combinational Circuits in Verilog

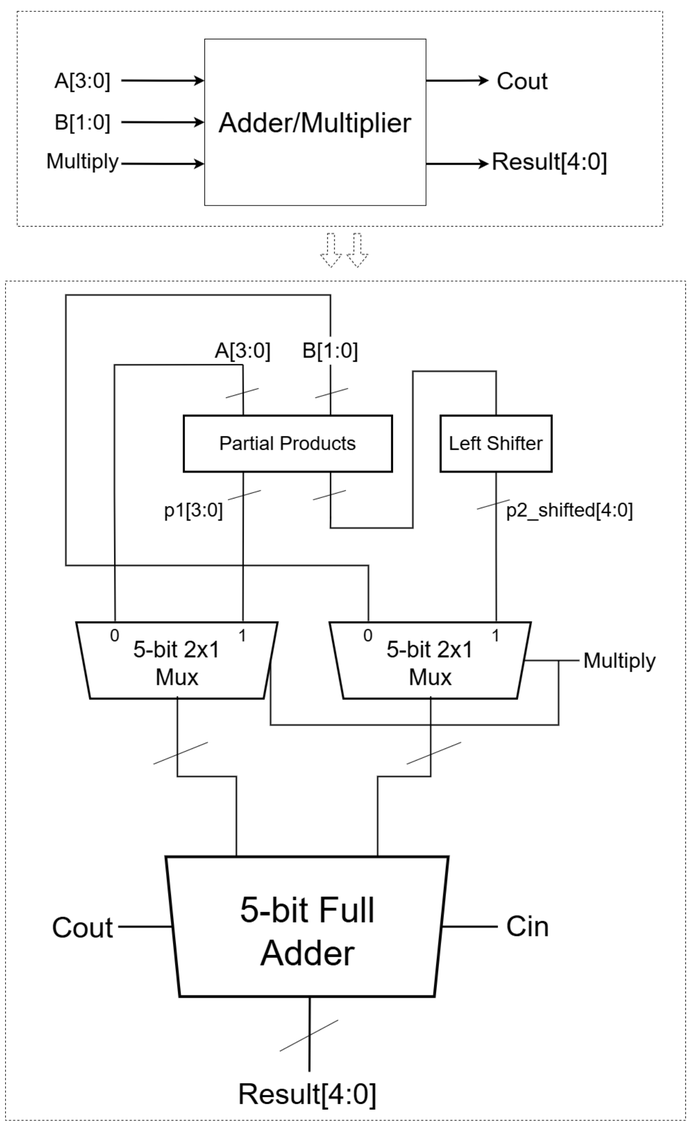
Due date: 02/12/2025, 23:59:59 **Via: https://submit.cs.hacettepe.edu.tr/**

BBM233 Digital Design Lab - 2025 Fall

Part 3 - Adder/Multiplier (30%)

Combinational logic circuits can be connected together to form larger circuits. This is done by taking outputs from one circuit and using them as inputs to another. **In the figure below, a circuit diagram of a 5-bit adder/mutiplier circuit implemented with the previously defined modules is given.**

Note: Some input bit sizes do not match the modules they are connected to. For instance, signal `B` is connected to a multiplexer whose input requires 5 bits. Therefore, 3 zero bits are added to the most significant side (to the left) of `B` to match the required size.



A **5-bit adder/multiplier** circuit has **one 4-bit and one 2-bit inputs** `A` and `B`, and a **1-bit input** `multiply`. `A` and `B` are the numbers that will be either added or multiplied, while `multiply` is the mode signal: when `multiply` is **1 (HIGH)**, multipication should be performed (**`Result = A x B`**), whereas when `multiply` is **0 (LOW)**, addition should be performed (**`Result = A + B`**).

This circuit has two outputs: one 5-bit output `Result` and **one 1-bit output** `Cout`. `Result` will output the result of the desired computation on the input numbers `A` and `B` (either their sum or multipication), whereas `Cout` will output any carry-out resulting from the calculation.

This circuit shows us that it is possible to implement a multiplier using an adder. The idea is to obtain `A x B` by actually left shifting of bits to calculate multipication and the rest is summation. The multiplexer serves as the selector circuit to pass either `A` or `p1` and `B` or `p2_shifter` to the adder based on the selected mode of operation via the signal `Multiply`.

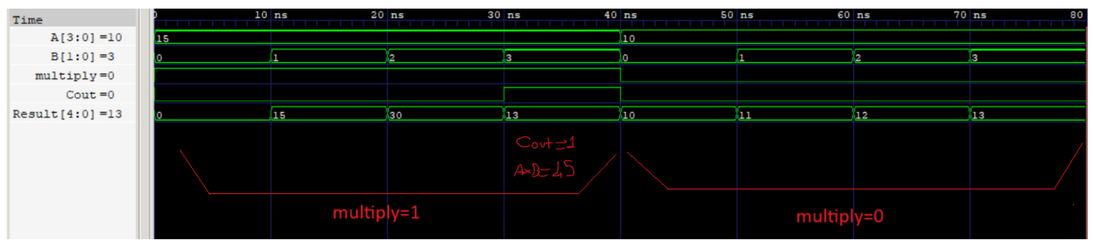
Part 3 - Adder/Multiplier (cont'd)

Experiment Steps

Implement the **5-bit adder/multiplier** circuit in Verilog using the circuit diagram on the left and the following steps:

1. Implement your module in Verilog while looking at the connections in the diagram on the left. Use **structural design approach and explicit association** to connect the relevant ports to each other. The module must be named as **`main.v`**.
2. Test it by writing an appropriate testbench **`main_tb.v`** for all possible input cases.
3. **You MUST download and use these starter code files before you start working! Do NOT change the I/O port names, order, or bit width!**

Make sure to obtain a similar waveform that shows the correctness of your design.



Automatic grading [VERY IMPORTANT!]

Your submissions will be graded using an automatic grading script. They **will not be** white-box tested. So, you are expected to **match the given wave forms 100%**, to receive full-credit.

You MUST download and use these starter code files before you start working! Do NOT change the I/O ports (names, order, bit width, etc.)!

Experiment 4 - Combinational Circuits in Verilog

Due date: 02/12/2025, 23:59:59 **Via: https://submit.cs.hacettepe.edu.tr/**

BBM233 Digital Design Lab - 2025 Fall

Testing your codes before submission

You can test your codes via our Tur^{6}Bo Grader before the submission to see which tests you are passing. **Note that this is used for testing purposes only**, and you still have to submit your full codes and report via https://submit.cs.hacettepe.edu.tr/ to get graded.

Submission via submit.cs

Submissions will be accepted via https://submit.cs.hacettepe.edu.tr/. **Note that the deadline for submission of the report and all codes is 02/12/2025, 23:59:59.**

Your submission will include the Verilog codes and a report PDF and it must be in the following format to be accepted:

- b<studentID>.zip
  - multiplier.v
  - multiplier_tb.v
  - five_bit_2x1_mux.v
  - five_bit_2x1_mux_tb.v
  - full_adder.v
  - full_adder_tb.v
  - five_bit_rca.v
  - five_bit_rca_tb.v
  - main.v
  - main_tb.v

Note that only ZIP archives are accepted!

Do not use any ready code from the internet as it will be considered as plagiarism if more than one student has the same solution even if they found the solutions individually.

Grading

Verilog codes: 100%

- Part 0: 25%
- Part 1: 20%
- Part 2: 25%
- Part 3: 30%